

Think Without Words: Recurrent Latent-Space Reasoning

DataForge 2026 Pathway Track - One-Page Technical Summary

Central claim. Recurrent computation can allocate additional inference work through repeated latent-state updates rather than additional verbal reasoning steps; in a controlled multi-step task, increasing the computation budget lets a fixed-size state progress further toward the target until sufficient depth is reached.

Problem and mechanism

Autoregressive reasoning systems commonly spend more test-time compute by generating more tokens, often as chain-of-thought (CoT). Recurrent latent-space reasoning explores a different allocation: an internal state is updated repeatedly without decoding every intermediate state into natural-language tokens. Coconut represents the reasoning state with a continuous hidden vector and feeds that state back as the next input; Geiping et al. study recurrent depth as a direct test-time compute knob. These works motivate the question our artifact teaches: what changes when the same state-update computation is allowed to run for more steps?

Interactive substrate

Our public demo is a controlled toy substrate, not a frontier language model. A synthetic task defines a deterministic sequence of state transitions, an initial state, and a known target after T transitions. The learner changes the recurrent computation budget (0-32 steps). The app shows the current state, ground-truth target, predicted state, and a 2D projection of the state trajectory. The population-level experiment evaluates thousands of tasks with different depths. Our measured curve is 0.0%, 12.0%, 24.0%, 49.3%, 74.3%, and 100.0% accuracy at budgets 0, 1, 2, 4, 6, and 8 respectively; accuracy remains 100% through 32 because the generated tasks require at most eight transitions. These results demonstrate the mechanics of computation depth in the toy environment, not superiority of a particular frontier model.

Architectural comparison

System	Where extra compute lives	State / memory	Observability
Transformer + CoT	More generated tokens	Token history / KV cache	High
Recurrent latent reasoning	Repeated hidden-state updates	Fixed-size latent state	Lower
BDH-CQ	Iterative latent reasoning	Recurrent memory updated from demonstrations	Lower; state-centric

BDH / BDH-CQ connection and evidence

BDH (Dragon Hatchling) is a brain-inspired post-Transformer architecture built around locally interacting neurons, sparse positive activations, and working memory implemented through synaptic plasticity. BDH-CQ is the direct connection for this project: its 2026 technical report combines inference-time demonstration updates to recurrent memory with iterative latent reasoning over a query, without verbalizing intermediate reasoning. The relationship is therefore architectural rather than superficial: our demo teaches recurrent state evolution; BDH-CQ adds an inference-time memory mechanism around recurrent latent reasoning. We explicitly label our memory widget as a simplified educational abstraction, not BDH-CQ internals. Reported evidence for BDH-CQ includes a 150M-parameter configuration achieving 29.5% pass@2 on ARC-AGI-1 at a computed inference cost of \$0.0007 per task; this is a developer-reported benchmark result at a specific operating point, not an independent proof of universal superiority.

Maturity, trade-offs, and limitations

Maturity assessment: 6/10. The research direction is active and has multiple primary studies, but it remains less mature than standard Transformer/CoT practice. Potential benefits include allocating compute in latent space, avoiding some verbalization overhead, and enabling reasoning dynamics that are awkward to serialize as words. Trade-offs include lower interpretability of intermediate states, evaluation difficulty, sensitivity to compute budget, and unclear transfer from controlled demonstrations to broad real-world reasoning. Our toy environment has an especially important limitation: it uses explicit deterministic transition dynamics rather than a learned language model, so its accuracy curve is evidence about the mechanism we constructed, not a direct benchmark claim about frontier systems.

Selected primary sources

1. Hao et al. (2024), *Training Large Language Models to Reason in a Continuous Latent Space* (Coconut), arXiv:2412.06769, <https://arxiv.org/abs/2412.06769>. 2. Geiping et al. (2025), *Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach*, NeurIPS 2025, arXiv:2502.05171, <https://arxiv.org/abs/2502.05171>. 3. Engdahl et al. (2026), *BDH-CQ: In-Context Learning with Recurrent Latent Reasoning*, arXiv:2608.09888, <https://arxiv.org/abs/2608.09888>. 4. Kosowski et al. (2025), *The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain*, arXiv:2509.26507, <https://arxiv.org/abs/2509.26507>.

Evidence boundary: live computation = toy recurrent-state simulator; aggregate curve = precomputed evaluation of the same simulator; BDH/BDH-CQ claims are sourced from the cited primary papers. The project does not present the toy system as an official BDH or BDH-CQ implementation.